

Timer

Timer

1. Learning Objectives
 - Timer
 - Timer Basic Parameters
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. Timer Parameter Configuration
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Learn the basic use of the timer on the MSPM0G3519 motherboard.
2. Implement timed LED status toggle.

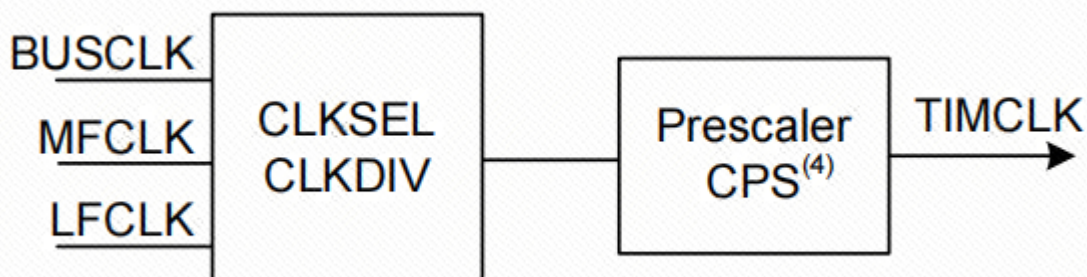
Timer

The timer is a very important peripheral in microcontrollers, used for timing and generating timed interrupts. The core principle of a timer is to record time through an internal counter. The counter increments by one for each machine cycle. When the counter reaches a preset value, it triggers an interrupt. The program can execute certain operations in the interrupt service routine, such as toggling an LED, reading sensor data, controlling a motor, etc.

Timer Basic Parameters

1. Prescaler

The prescaler can divide the timer clock (TIMCLK) frequency by any value between 1 and 256 (1 to 256 based on the timer prescaler being 8-bit). TIMG can select BUSCLK, MFCLK, or LFCLK as the clock source. It can divide the clock by up to 8, then passes through an 8-bit prescaler to become the timer's counting clock.



2. Up or Down Counting Mode

Up counting means the counter continuously counts up from 0 to the auto-reload value. Once the counter counts to the auto-reload value, it resets to 0 and continues counting up, generating an overflow event. Down counting means the counter continuously counts down from the auto-reload value to 0. Once the counter counts to 0, it resets to the auto-reload value and continues counting down, generating an underflow event.

3. Update Event

An update event is triggered when the counter starts a new counting cycle due to overflow or underflow. Update events can trigger DMA requests to timely update the timer's operating parameters at the start of the next counting cycle, which is especially suitable for real-time control.

2. Hardware Setup

The MSPM0G series has 7 timers in total, which can be divided into 2 types: general-purpose timers (TIMG) and advanced control timers (TIMA). Different types of timers have different numbers of features. Generally, advanced timers have the most features, and general-purpose timers have fewer.

Table 8-15. TIMx Configurations

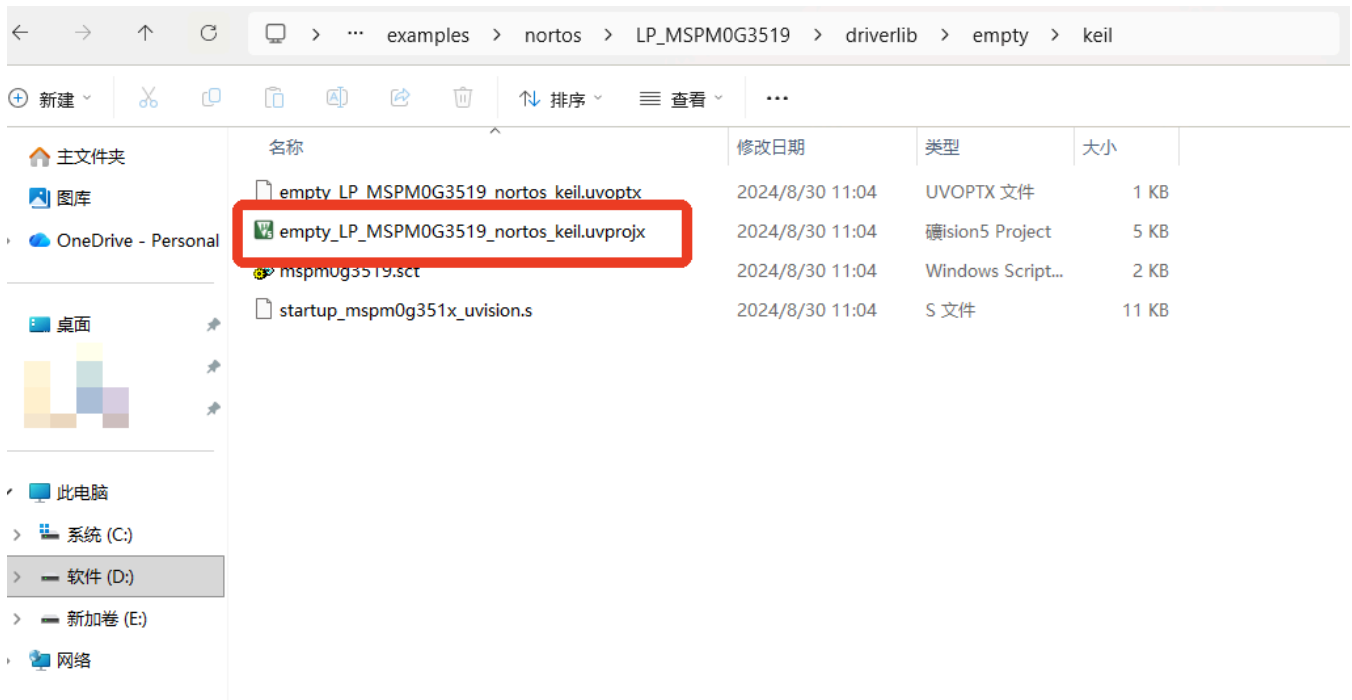
TIMER NAME	POWER DOMAIN	RESOLUTION	PRESCALE R	REPEAT COUNTER	CAPTURE / COMPARE CHANNELS	PHASE LOAD	SHADOW LOAD	SHADOW CC	DEADBAND	FAULT	QEI
TIMG0	PD0	16 bit	8 bit	–	2	–	–	–	–	–	–
TIMG6	PD1	16 bit	8 bit	–	2	–	–	–	–	–	–
TIMG7	PD1	16 bit	8 bit	–	2	–	Yes	Yes	–	–	–
TIMG8	PD0	16 bit	8 bit	–	2	–	–	–	–	–	Yes
TIMG9	PD0	16 bit	8 bit	–	2	–	–	–	–	–	Yes
TIMG12	PD1	32 bit	–	–	2	–	–	Yes	–	–	–
TIMG14	PD0	16 bit	8 bit	–	4	–	–	–	–	–	–
TIMA0	PD1	16 bit	8 bit	8 bit	4	Yes	Yes	Yes	Yes	Yes	–
TIMA1	PD1	16 bit	8 bit	8 bit	2	Yes	Yes	Yes	Yes	Yes	–

3. Experiment Steps

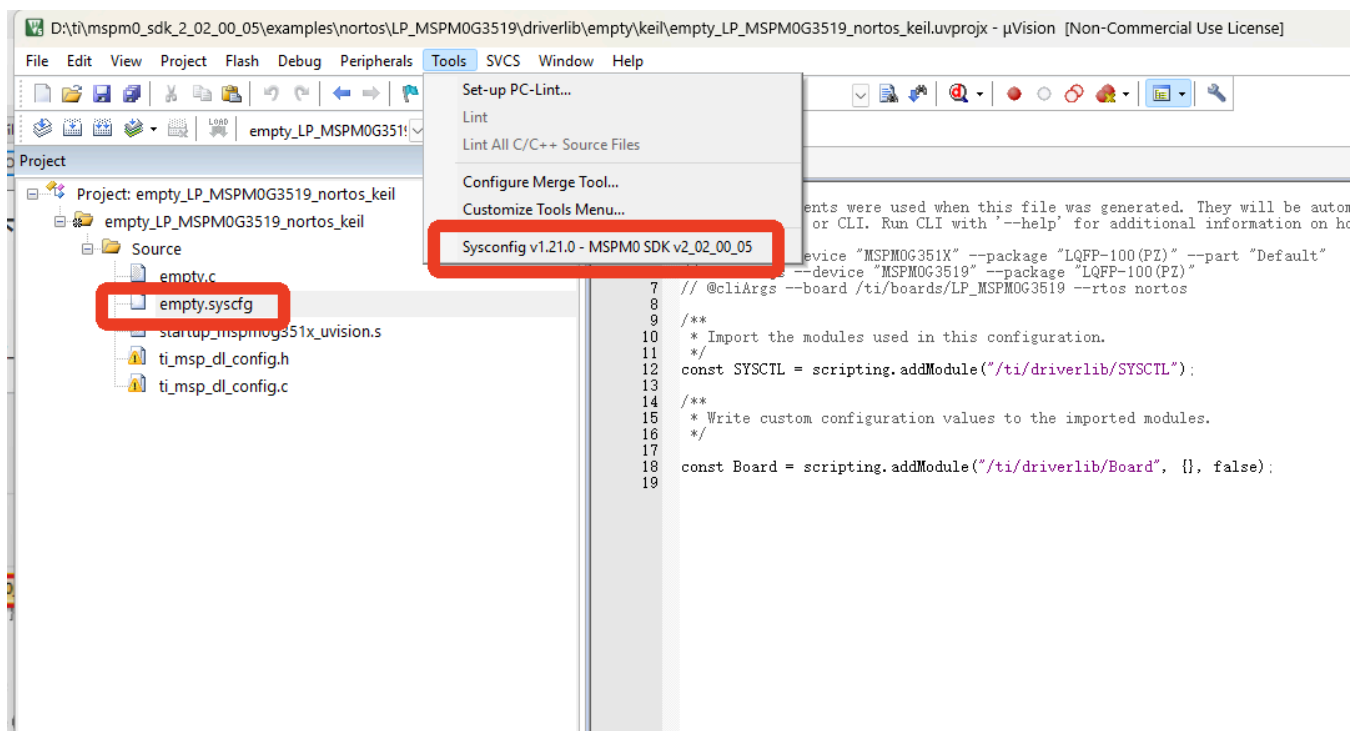
If a project needs to execute or repeatedly execute specific tasks at regular intervals, timer configuration is particularly important. The following will introduce how to configure the timer to trigger an interrupt every 1 second and implement LED status toggle in the interrupt service function, thereby achieving the effect of LED on for 1 second and off for 1 second.

1. Open SYSCONFIG Configuration Tool

Open the empty project from the SDK in KEIL.



After selecting and opening it, open the empty.syscfg file in the KEIL interface. **With the empty.syscfg file open**, then open the SYSCONFIG GUI interface from the Tools menu.



In SYSCONFIG, select MCU peripherals on the left, find the TIMER option and click to enter. Click ADD in TIMER to add a timer peripheral.

The screenshot displays the TI Studio IDE interface for configuring a timer. The left sidebar shows the project tree with 'TIMER' highlighted (1). The main window shows the 'TIMER (0 of 9 Added)' configuration page with an 'ADD' button highlighted (2). The right sidebar shows the 'Generated Files' and 'MSPM0G3519 (Device)' sections.

Generated Files:

File name	Category	Include in build
ti_msp_dl_config.c	MSPM0 Driver Library	☒
ti_msp_dl_config.h	MSPM0 Driver Library	☒
Event.dot	MSPM0 Driver Library	☒
empty.syscfg	Configuration Script	☐

MSPM0G3519 (Device) LQFP-100(PZ) (Package)

The package pinout shows pins 1 through 21 on the left and 75 through 55 on the right. Pin 1 is highlighted in red.

2. Timer Parameter Configuration

In this case, Timer Clock Source selects BUSCLK-32MHz. Timer Clock Divider configures timer division, selecting divide by 8. Timer Clock Prescaler configures 100 prescaler, resulting in a timer frequency of 40KHz. Desired Timer Period sets the timer period to 1S. Timer Mode sets to periodic down-counting mode. Enable Interrupts enables the 0 overflow interrupt. LED GPIO configuration follows the LED light course.

TIMER (1 of 7 Added) ?

+

ADD

≡

REMOVE ALL

✓

TIMER_0

📄

🗑️

Name

TIMER_0

Selected Peripheral

TIMG0

Quick Profiles

Timer Profiles

Custom

Basic Configuration

Clock Configuration

Timer Clock Source

BUSCLK

Timer Clock Divider

Divided by 8

Calculated Timer Clock Source

4000000

Timer Clock Prescaler

100

Calculated Timer Clock Values

Timer Clock Frequency

40.00 kHz

Timer Period Range And Resolution

25.00 μ s to 1.64 s w/ resolution of 25.00 μ s

Timer Mode

Periodic Down Counting

Desired Timer Period

1000 ms

Actual Timer Period

1.00 s

Start Timer ?

☒

Advanced Configuration

Interrupts Configuration

Enable Interrupts

Zero event

Interrupt Priority

Default

Event Configuration

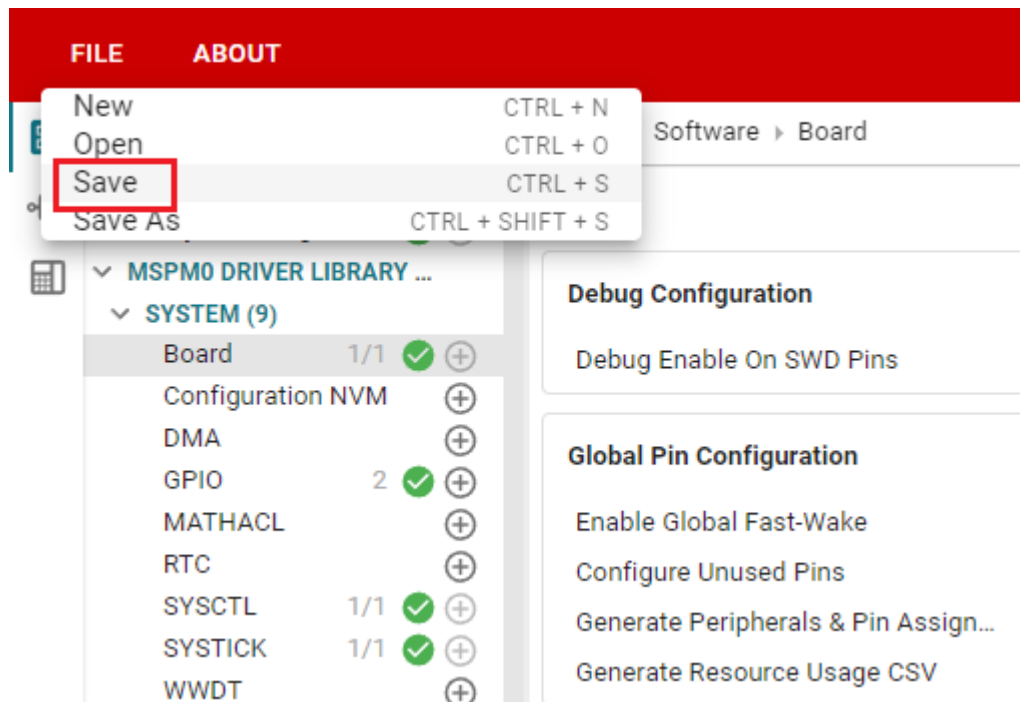
PinMux Peripheral and Pin Configuration

Timer Peripheral

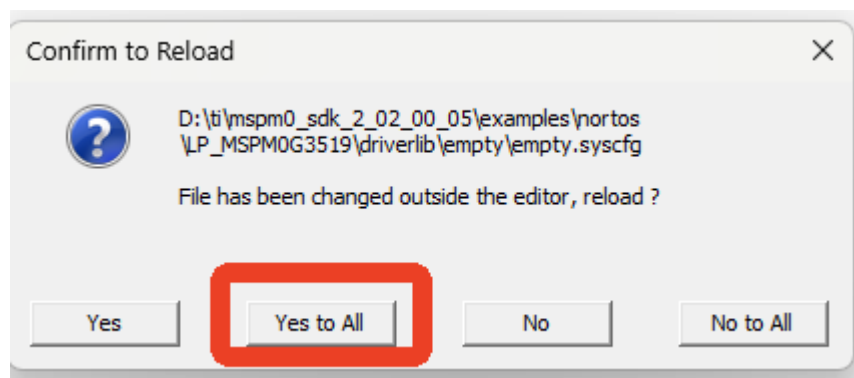
TIMG0

🔒

Click SAVE to save the SYSCONFIG configuration, then close SYSCONFIG and return to KEIL.



Select **Yes to All** in the popup.



We also need to confirm whether `ti_msp_dl_config.c` and `ti_msp_dl_config.h` files are updated. Simply compile, and the compilation will automatically update to KEIL. If not updated, we can also copy the file content from `SYSCONFIG`.

3. Write Program

In the `empty.c` file, write the following code

```
#include "ti_msp_dl_config.h"

int main(void)
{
    SYSCFG_DL_init();
    // Clear the timer interrupt flag
    NVIC_ClearPendingIRQ(TIMER_0_INST_INT_IRQN);
    // Enable timer interrupt
    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);

    while (1)
    {
```

```

    }
}

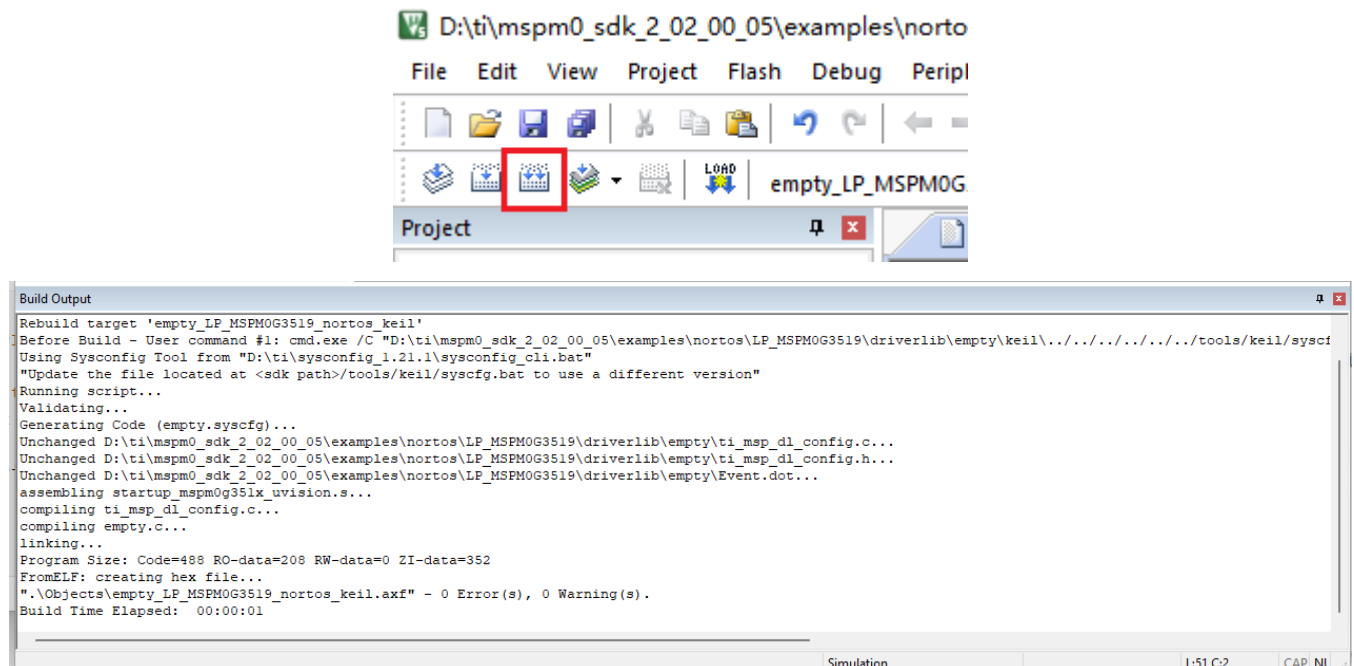
// The timer interrupt service function has been configured to a period of 1 second
void TIMER_0_INST_IRQHandler(void)
{
    // If a timer interrupt occurs
    switch( DL_TimerG_getPendingInterrupt(TIMER_0_INST) )
    {
        case DL_TIMER_IIDX_ZERO:// If it is 0 overflow interrupt
            // Flip the LED status
            DL_GPIO_togglePins(LED1_PORT, LED1_PIN_2_PIN);
            break;

        default:// other timer interrupts
            break;
    }
}
}

```

4. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



4. Program Analysis

- empty.c

```

// The timer interrupt service function has been configured to a period of 1 second
void TIMER_0_INST_IRQHandler(void)
{
    // If a timer interrupt occurs
    switch( DL_TimerG_getPendingInterrupt(TIMER_0_INST) )
    {
        case DL_TIMER_IIDX_ZERO:// If it is 0 overflow interrupt

```

```
        // Flip the LED status
        DL_GPIO_togglePins(LED1_PORT, LED1_PIN_2_PIN);
        break;

    default:// Other timer interrupts
        break;
}
}
```

This is a timer interrupt service routine used to handle timer overflow interrupts and execute corresponding operations. Whenever the timer overflows, it toggles the status of an LED.

5. Experimental Phenomena

After the program is downloaded, you can see the LED light flashing according to the 1-second period configuration.